Conexión ESP32 ↔ App móvil

Documento técnico de la comunicación entre el **ESP32** (lectura de sensores MAX30102 y MLX90614) y la **aplicación móvil** (React Native / Expo), describiendo cada capa de comunicación, los protocolos implicados, el formato de los datos y el código que los implementa.

---

1. Resumen

El ESP32 actúa como **servidor WebSocket**. La app móvil actúa como **cliente WebSocket**. Ambos están conectados a la **misma red WiFi (LAN)**. El ESP32 mide los sensores y **difunde (broadcast)** un objeto **JSON** por cada ciclo de medición (≈1 vez por segundo). La app móvil escucha ese canal y actualiza la interfaz en tiempo real.



- **Medio físico / enlace:** WiFi 802.11 (2.4 GHz).
- **Transporte:** TCP.
- **Aplicación:** WebSocket (RFC 6455) con mensajes de texto en **JSON**.
- **Sensores:** bus **I^2C** (interno al ESP32).

---

2. Capas de comunicación

Capa (modelo OSI/TCP-IP)	Protocolo	Quién	Detalle
Física (sensores)	I^2C	ESP32 ↔ MAX30102 / MLX90614	SDA=GPIO21, SCL=GPIO22, 100 kHz
Física/Enlace (red)	IEEE 802.11 (WiFi)	ESP32 ↔ Router	2.4 GHz, modo STA
Red	IPv4 + DHCP	ESP32 obtiene IP del router	Misma subred que el móvil
Transporte	TCP	ESP32 ↔ móvil	Puerto **81**

Capa (modelo OSI/TCP-IP)	Protocolo	Quién	Detalle
Aplicación	**WebSocket (RFC 6455)**	ESP32 (servidor) ↔ móvil (cliente)	Handshake HTTP + frames
Formato de datos	**JSON (UTF-8)**	ESP32 → móvil	`broadcastTXT()` / `event.data`

> Nota: el path de la app móvil **no** usa HTTP de forma directa para los datos; solo se usa HTTP durante el *handshake* inicial del WebSocket (upgrade) y luego el canal es WebSocket puro.

---

3. Capa de sensores: bus I²C

Los sensores no se comunican por WiFi; van cableados al ESP32 por **I²C**. La configuración está en `esp32/esp32_sensores/esp32_sensores.ino`:

```
#define SDA_PIN 21
#define SCL_PIN 22
...
Wire.begin(SDA_PIN, SCL_PIN);
Wire.setClock(100000);   // 100 kHz (modo estandar)
```

- **MAX30102** (oxímetro/pulsómetro): dirección I²C 7-bit `0x57`. Se inicializa con `max30102.begin(Wire, I2C_SPEED_STANDARD)`.
- **MLX90614** (termómetro IR): dirección I²C 7-bit `0x5A`. Se inicializa con `mlx90614.begin(0x5A, &Wire)`.

Parámetros del MAX30102:

```
byte ledBrightness = 60;
byte sampleAverage = 4;   // promedia 4 muestras -> salida efectiva ~25 Hz
byte ledMode = 2;         // LED rojo + infrarrojo
int sampleRate = 100;     // 100 muestras/s en el FIFO
int pulseWidth = 411;
int adcRange = 4096;
```

Con `sampleRate = 100` y `sampleAverage = 4`, la FIFO entrega ~25 muestras/seg. El `loop()` consume **25 muestras** por iteración, por lo que cada ciclo dura aproximadamente **1 segundo** — de ahí la cadencia de publicación.

---

4. Capa de red: WiFi

El ESP32 se conecta en **modo estación (STA)** y obtiene una IP por **DHCP**:

```
void conectarWiFi() {
  ...
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);

  unsigned long inicio = millis();
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    if (millis() - inicio > 20000) {      // reintento cada 20 s
      WiFi.disconnect();
      WiFi.begin(WIFI_SSID, WIFI_PASS);
      inicio = millis();
    }
  }

  Serial.print("IP del ESP32: ");
  Serial.println(WiFi.localIP());         // <-- IP que se pone en la app
}
```

En cada iteración del `loop()` se verifica que la conexión siga activa y, si se pierde, se reconecta:

```
void loop() {
  webSocket.loop();
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi perdido. Reconectando...");
    conectarWiFi();
  }
  ...
}
```

Requisito crítico: ESP32 y teléfono deben estar en la **misma red/subred** para que la IP del ESP32 sea alcanzable. La IP se muestra en el monitor serie y se escribe en la pantalla **Configuración** de la app.

Conexión lógica resultante:

```
App móvil ──(WiFi 802.11)── Router ──(WiFi 802.11)── ESP32
                    │
                    └─ mismos 192.168.x.0/24
```

4.1 Direccionamiento y puerto

La URL que usa el cliente es:

```
ws://<IP_DEL_ESP32>:81
```

- `ws://` = WebSocket **sin TLS** (texto plano). Ver sección 9.
- `<IP_DEL_ESP32>` = IP obtenida por DHCP (ej. `192.168.1.50`).
- `:81` = puerto TCP del servidor. Se eligió 81 para evitar el 80 (habitual en HTTP).

En el ESP32 el servidor se crea así (`esp32_sensores.ino:49`):

```
WebSocketsServer webSocket = WebSocketsServer(81);
```

Y en la app móvil (`mobile-app/src/lib/useEsp32.js:32`):

```
ws = new WebSocket(`ws://${ip}:${port}`);   // port = 81 por defecto
```

---

5. Protocolo: WebSocket (RFC 6455)

WebSocket aporta comunicación **bidireccional y full-duplex** sobre una única conexión TCP. El flujo es:

5.1 Handshake (HTTP Upgrade)

El cliente (app móvil) abre TCP contra `IP:81` y envía una petición HTTP de *upgrade*:

```
GET / HTTP/1.1
Host: 192.168.1.50:81
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: <clave aleatoria base64>
Sec-WebSocket-Version: 13
```

El servidor (ESP32, librería *WebSockets* de Markus Sattler) responde:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: <SHA-1(clave + GUID) base64>
```

Tras el `101 Switching Protocols`, el canal HTTP deja de existir y **todos** los mensajes siguientes son **frames WebSocket**. Es en este punto cuando el `onopen` del cliente se dispara.

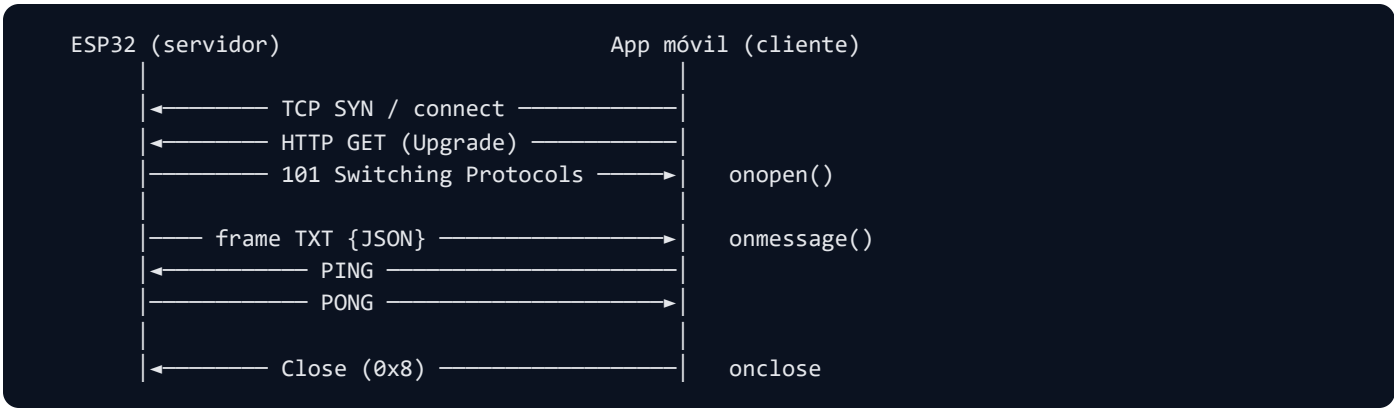
5.2 Frames

Cada mensaje viaja como un *frame* binario con cabecera de 2–14 bytes:

Campo	Descripción
FIN	1 = último fragmento del mensaje
opcode	`0x1` texto, `0x2` binario, `0x8` close, `0x9` ping, `0xA` pong
MASK	clientes→servidor van enmascarados (obligatorio por el RFC)
Payload length	7/16/64 bits según tamaño
Payload	datos del mensaje

- El ESP32 envía **frames de texto** (opcode `0x1`) con el JSON.
- La librería gestiona internamente **ping/pong** y **close**.
- `broadcastTXT()` envía el mismo frame texto a **todos** los clientes conectados.

5.3 Modelo de comunicación



> El flujo de datos es principalmente **ESP32 → app móvil** (telemetría). La app no necesita enviar comandos; el canal cliente→servidor solo se usa para control (ping/pong/close).

---

6. Lado ESP32 (servidor)

6.1 Creación y arranque

```
WebSocketsServer webSocket = WebSocketsServer(81);

void setup() {
  ...
  webSocket.begin();                    // abre el puerto 81 y escucha
  webSocket.onEvent(onWebSocketEvent);  // registra el callback de eventos
}
```

6.1.1 Manejador de eventos

La librería invoca un callback por cada evento del cliente:

```
void onWebSocketEvent(uint8_t client, WStype_t type, uint8_t* payload, size_t length) {
  switch (type) {
    case WStype_CONNECTED: {
      IPAddress ip = webSocket.remoteIP(client);
      Serial.printf("[WS] Cliente #%u conectado desde %u.%u.%u.%u\n",
                    client, ip[0], ip[1], ip[2], ip[3]);
      break;
    }
    case WStype_DISCONNECTED:
      Serial.printf("[WS] Cliente #%u desconectado\n", client);
      break;
    case WStype_TEXT:
      break;   // la app no envia datos obligatorios; se ignora
    default:
      break;
  }
}
```

Eventos relevantes de la librería: `WStype_CONNECTED`, `WStype_DISCONNECTED`, `WStype_TEXT`, `WStype_BIN`, `WStype_PING`, `WStype_PONG`, `WStype_ERROR`.

6.2 Bucle de servicio (obligatorio)

`webSocket.loop()` procesa la E/S del socket (aceptar clientes, leer frames, responder ping/pong, cerrar conexiones). **Debe llamarse con frecuencia**:

```
void loop() {
  webSocket.loop();
  ...
}
```

Como el código **bloquea** a la espera de 25 muestras del sensor, también se llama `webSocket.loop()` dentro de esos bucles de espera para no dejar "colgado" el canal:

```
for (int i = 75; i < BUFFER_SIZE; i++) {
  while (!max30102.available()) {
    max30102.check();
    webSocket.loop();     // <- mantiene vivo el WebSocket mientras se espera
  }
  ...
}
```

6.3 Construcción y difusión del JSON

El payload se arma como una cadena JSON (sin librería externa, para minimizar dependencias):

```
String construirJSON(bool dedo, int32_t hr, bool hrValido,
                     int32_t sp, bool spValido,
                     float tempObjeto, float tempAmbiente,
                     uint32_t ir, uint32_t red) {
  String json = "{";
  json += "\"finger\":" + String(dedo ? "true" : "false");
  json += ",\"bpm\":"   + String(hrValido ? String(hr) : String(0));
  json += ",\"spo2\":"  + String(spValido ? String(sp) : String(0));
  json += ",\"validBpm\":"  + String(hrValido ? "true" : "false");
  json += ",\"validSpo2\":" + String(spValido ? "true" : "false");
  json += ",\"temp\":"    + String(tempObjeto, 2);
  json += ",\"tempAmb\":" + String(tempAmbiente, 2);
  json += ",\"ir\":"  + String(ir);
  json += ",\"red\":" + String(red);
  json += ",\"ms\":"  + String(millis());
  json += "}";
  return json;
}
```

Y en el `loop()`, al terminar cada ciclo de medición:

```
String json = construirJSON(
  dedoDetectado,
  heartRate, hrValido && dedoDetectado,
  spo2,      spValido && dedoDetectado,
  temperaturaObjeto, temperaturaAmbiente,
  valorIR, valorRojo
);

webSocket.broadcastTXT(json);
```

`broadcastTXT` envía el JSON como frame de **texto** a todos los clientes conectados. Si no hay clientes, no se envía a nadie (sin error).

6.3.1 Cadencia de envío

- `sampleRate = 100`, `sampleAverage = 4` → FIFO a ≈25 Hz.

- El `loop()` toma **25 muestras** nuevas por iteración → **≈1 broadcast por segundo**.
- Además, el tiempo de cómputo del algoritmo `maxim_heart_rate_and_oxygen_saturation()` y la lectura del MLX90614 se suman al ciclo.

6.4 Ciclo de medición (resumen del `loop()`)

```
1. webSocket.loop()                -> atender red
2. ¿WiFi caído? -> reconectar
3. Desplazar buffer 25 posiciones (conservar las últimas 75 muestras)
4. Adquirir 25 muestras nuevas sensor MAX30102 (webSocket.loop() durante espera)
5. Calcular BPM y SpO2 (maxim_heart_rate_and_oxygen_saturation)
6. Leer temperatura objeto y ambiente (MLX90614)
7. Validar rangos (hrValido, spValido) y detectar dedo
8. Construir JSON y webSocket.broadcastTXT(json)
```

---

7. Capa de la app móvil (cliente WebSocket)

Todo el cliente está encapsulado en el hook `useEsp32(ip, port = 81)` (`mobile-app/src/lib/useEsp32.js`). El estado que expone es `{ status, reading, lastAt }`.

7.1 Creación del cliente y eventos

```
const ws = new WebSocket(`ws://${ip}:${port}`);
wsRef.current = ws;

ws.onopen = () => {
  if (!stopped) setStatus("connected");
};

ws.onmessage = (event) => {
  if (stopped) return;
  try {
    const data = JSON.parse(event.data);   // el ESP32 manda texto JSON
    setReading(data);
    setLastAt(Date.now());
    setStatus("connected");
  } catch (e) {
    // ignora mensajes no-JSON
  }
};

ws.onerror = () => { if (!stopped) setStatus("error"); };

ws.onclose = () => {
  if (stopped) return;
  setStatus("disconnected");
  scheduleReconnect();
};
```

- `onopen` ↔ handshake WebSocket completado (HTTP 101).
- `onmessage` ↔ cada frame de texto recibido (JSON del ESP32).
- `onclose` ↔ conexión cerrada (por red o por el servidor).

7.2 Reconexión automática

Como una LAN/power-loss puede tumbar el socket, el cliente reintenta cada 3 s:

```
const scheduleReconnect = () => {
  if (stopped) return;
  reconnectTimer = setTimeout(connect, 3000);
};
```

7.3 Ciclo de vida y limpieza

El hook se re-ejecuta cuando cambian `ip` o `port`. En el *cleanup* se detiene la reconexión y se cierra el socket sin disparar `onclose`:

```
return () => {
  stopped = true;
  if (reconnectTimer) clearTimeout(reconnectTimer);
  if (wsRef.current) {
    wsRef.current.onclose = null;   // evita reconectar al desmontar
    wsRef.current.close();
    wsRef.current = null;
  }
};
```

7.4 Estados del cliente (`status`)

Estado	Significado
`idle`	Sin IP configurada
`connecting`	Intentando abrir el WebSocket
`connected`	Handshake OK y/o recibiendo datos
`error`	Fallo al abrir/mantener el socket
`disconnected`	Socket cerrado, se programó reconexión

---

8. Formato del mensaje (contrato de datos)

Cada frame de texto contiene un **objeto JSON**. Esquema:

Campo	Tipo	Unidad	Descripción
`finger`	bool	—	`true` si hay dedo detectado (IR > umbral)
`bpm`	int	lpm	Frecuencia cardiaca. `0` si no es válida
`spo2`	int	%	Saturación de oxígeno. `0` si no es válida
`validBpm`	bool	—	`bpm` está en rango válido (30–220)
`validSpo2`	bool	—	`spo2` está en rango válido (1–100)
`temp`	float(2)	°C	Temperatura objeto (MLX90614)
`tempAmb`	float(2)	°C	Temperatura ambiente (MLX90614)
`ir`	uint32	—	Valor crudo del LED infrarrojo
`red`	uint32	—	Valor crudo del LED rojo

Campo	Tipo	Unidad	Descripción
`ms`	uint32	ms	`millis()` del ESP32 en el momento del envío

Ejemplo real de frame:

```
{"finger":true,"bpm":72,"spo2":98,"validBpm":true,"validSpo2":true,"temp":36.52,"tempAmb":24.31,"ir":852
```

> La app interpreta `validBpm` / `validSpo2` como verdadero **solo si además hay dedo** (el firmware envía `hrValido && dedoDetectado`).

---

9. Seguridad

- El canal es `ws://` (sin cifrado TLS). En una LAN doméstica es aceptable, pero **cualquiera en la misma red** podría leer los frames.
- No hay autenticación en el WebSocket: el ESP32 acepta cualquier cliente.
- **Android / build de producción:** las apps nativas bloquean tráfico en claro. En **Expo Go** funciona; para APK/AAB:
  - añadir el plugin `expo-build-properties` con `android.usesCleartextTraffic: true`, **o**
  - usar TLS en el ESP32 con `wss://` (requiere certificado, ej. `WiFiClientSecure`).
- Endurecimiento recomendado: red WiFi dedicada, o WebSocket sobre TLS + token de sesión.

---

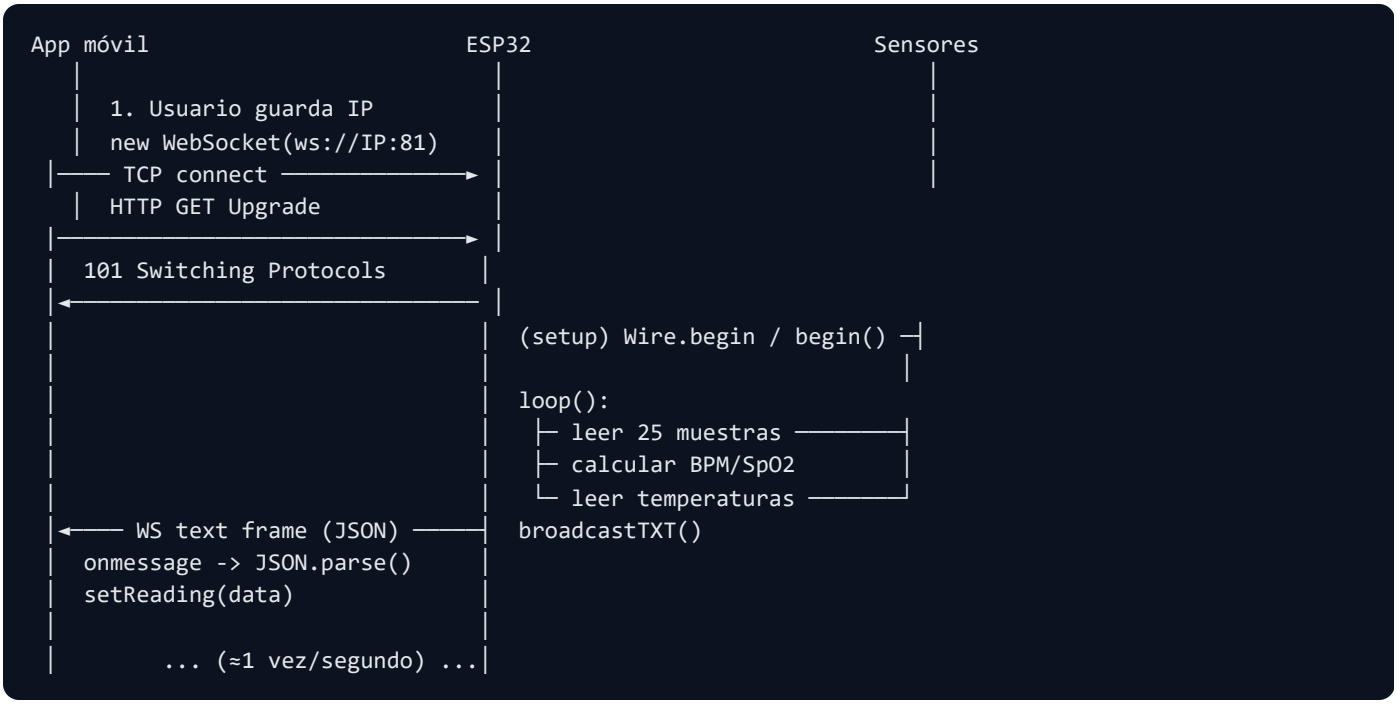
10. Diagnóstico de fallos

Síntoma	Causa probable	Solución
`status = idle`	No hay IP guardada	Escribir la IP en **Configuración**
`status = connecting` indefinido	IP/puerto incorrectos o distinta red	Verificar IP en monitor serie; misma WiFi
`status = error`	Puerto 81 bloqueado / firewall	Revisar router; usar otra red
`status = disconnected` en bucle	WiFi del ESP32 caído	Revisar alimentación/WiFi del ESP32
Datos en `--`	Sin dedo o `valid*` en falso	Colocar el dedo en el MAX30102
Temperatura errática	Sensor MLX90614 lejos/objeto	Corregir distancia y campo de visión

Comprobación rápida del canal desde una PC (con `websocat` o similar):

```
websocat ws://192.168.1.50:81
# debe empezar a imprimir un JSON por segundo
```

11. Diagrama de secuencia (extremo a extremo)



12. Referencias de archivos

Archivo	Rol
`esp32/esp32_sensores/esp32_sensores.ino`	Firmware: I²C, WiFi, servidor WebSocket, JSON
`mobile-app/src/lib/useEsp32.js`	Cliente WebSocket (hook), reconexión, parseo JSON
`mobile-app/src/screens/MonitorScreen.js`	Visualización de los datos recibidos
`mobile-app/src/lib/useFirebaseSync.js`	Envío posterior a Firebase (fuera del alcance de este documento)

Librerías usadas

- ESP32: `WebSockets` (Markus Sattler, links2004), `MAX30105` (SparkFun), `Adafruit MLX90614`.
- App móvil: API `WebSocket` nativa de React Native (no requiere librería extra).